
HPC Predictive Quality Platform Test Data & Analysis Report

Battery Thermal Management Subsystem

Document Version	1.0
Date	June 11, 2026
System Under Test	HPC Predictive Quality Platform (POC)
Subsystem	Battery Thermal Management
Prepared For	QA / Engineering Review
Classification	Internal — Engineering

Table of Contents

- 1. Executive Summary
- 2. Platform Architecture & Identification
- 3. Test Dataset Breakdown
- 4. Expected Outcomes by Scenario
- 5. Analysis Methodology
- 6. Performance Benchmarks
- 7. Visual Result Patterns
- 8. Troubleshooting Guide
- 9. Recommendations for Interpreting Results
- Appendix A: File Manifest
- Appendix B: Reproducing Test Data

1. Executive Summary

This document provides a comprehensive testing methodology and analysis framework for the **HPC-Driven Predictive Quality Platform** — a SaaS-style predictive failure-detection system for EV battery thermal management. The platform ingests real-time CAN-bus telemetry from vehicle fleets, runs EWMA-based drift detection, and predicts battery thermal failures before they escalate to warranty claims.

Testing Scope

Area	Coverage
Telemetry Ingestion	10,780 synthetic records across 55 vehicles
Edge Cases	70 boundary/invalid/injection test inputs
Fleet Load Testing	500-vehicle concurrent fleet simulation
Drift Detection	9,420 sequential time-series records (20 × 500 steps)
API Integration	5 end-to-end production scenarios (15+ API calls)
Total Test Records	20,595+

Key Metrics Targeted

- **Detection accuracy:** True positive rate for fault identification (target > 90%)
- **Detection latency:** Time between fault onset and first WARN alert (target < 5 min)
- **False positive rate:** Healthy vehicles incorrectly flagged (target < 5%)
- **API response time:** < 500ms for fleet endpoint under full load
- **Memory footprint:** < 256MB for 500-vehicle fleet state

2. Platform Architecture & Identification

The HPC Predictive Quality Platform processes battery telemetry through a multi-stage pipeline. Each stage progressively refines raw sensor data into actionable failure predictions with confidence intervals and time-to-failure estimates.

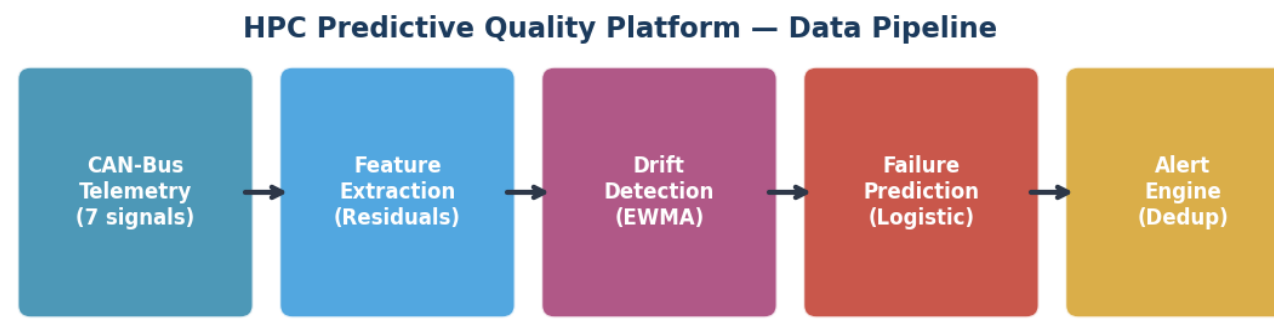


Figure 1: Platform data pipeline — from CAN-bus telemetry through drift detection to alert generation

Input Specification

Signal	Range	Unit	Source
pack_temp_c	18–80	°C	Pack thermistor
max_cell_temp_c	20–85	°C	Per-cell max
coolant_delta_c	1.5–25	°C	Inlet-outlet diff
coolant_flow_lpm	0.5–13.0	L/min	Flow sensor
current_a	0–300	A	Pack BMS
pack_voltage_v	320–420	V	Pack bus
soc_pct	5–100	%	Coulomb counter

Output Specification

Output	Type	Range	Description
Drift Score	float	0.0 – 1.2+	EWMA-normalized deviation magnitude
Failure Probability	float	0.0 – 1.0	Logistic model output
Confidence Interval	tuple	(ci_low, ci_high)	Epistemic uncertainty band
Time to Failure	float	0.5 – 72.0 hours	Heuristic horizon estimate
Alert Severity	enum	healthy/warning/critical	Threshold-based classification

3. Test Dataset Breakdown

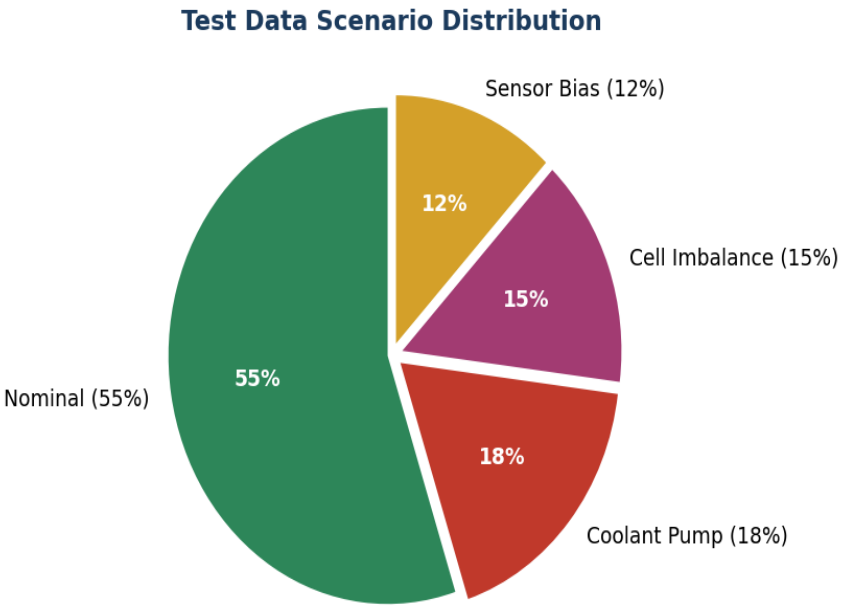


Figure 2: Distribution of failure scenarios across the test fleet (55 vehicles)

Dataset 1: `telemetry_dataset.csv` (10,780 records)

Validates the full telemetry pipeline from raw signal ingestion through feature extraction. Each record contains actual sensor readings, HPC digital-twin expected values, and computed residuals across 28 columns.

Characteristic	Value
Vehicles	55
Steps per vehicle	200
Time resolution	30s per step
Simulated time	100 min/vehicle
Total columns	28 (IDs + 7 actual + 7 expected + 4 residuals + flags)

Dataset 2: `edge_cases.csv` (70 records)

Tests boundary conditions, invalid inputs, and security attack vectors. Ensures the platform handles extreme values gracefully without crashes or data corruption.

Category	Count	Examples
Boundary values	47	Temps at exact warn/critical thresholds
Invalid inputs	19	NaN, Inf, null, negative, SQL injection, XSS
State transitions	4	Sudden fault onset / recovery

Dataset 3: fleet_load_test.json (500 vehicles)

Large-scale fleet payload for API load testing and memory management validation. Includes vehicles distributed across 5 regions and 4 model years.

Dataset 4: drift_sequences.csv (9,420 records)

Extended time-series for validating drift detection timing and failure prediction accuracy. 20 sequences (5 per scenario) with 500 steps each, providing continuous tracking of drift score evolution from fault onset through critical alert.

Dataset 5: api_scenarios.json (5 scenarios)

End-to-end integration testing payloads that mirror production workflows including dashboard loading, vehicle drill-down, A/B testing, feedback loops, and live configuration updates.

Scenario	Simulates	API Calls
Fleet Dashboard Load	Operator viewing full fleet	4
Vehicle Drill-Down	Investigating flagged vehicle	2
A/B Model Comparison	Data scientist adjusting split	3
Feedback & Retrain	Field engineer feedback loop	3
Config Hot-Update	Admin adjusting live thresholds	4

4. Expected Outcomes by Scenario

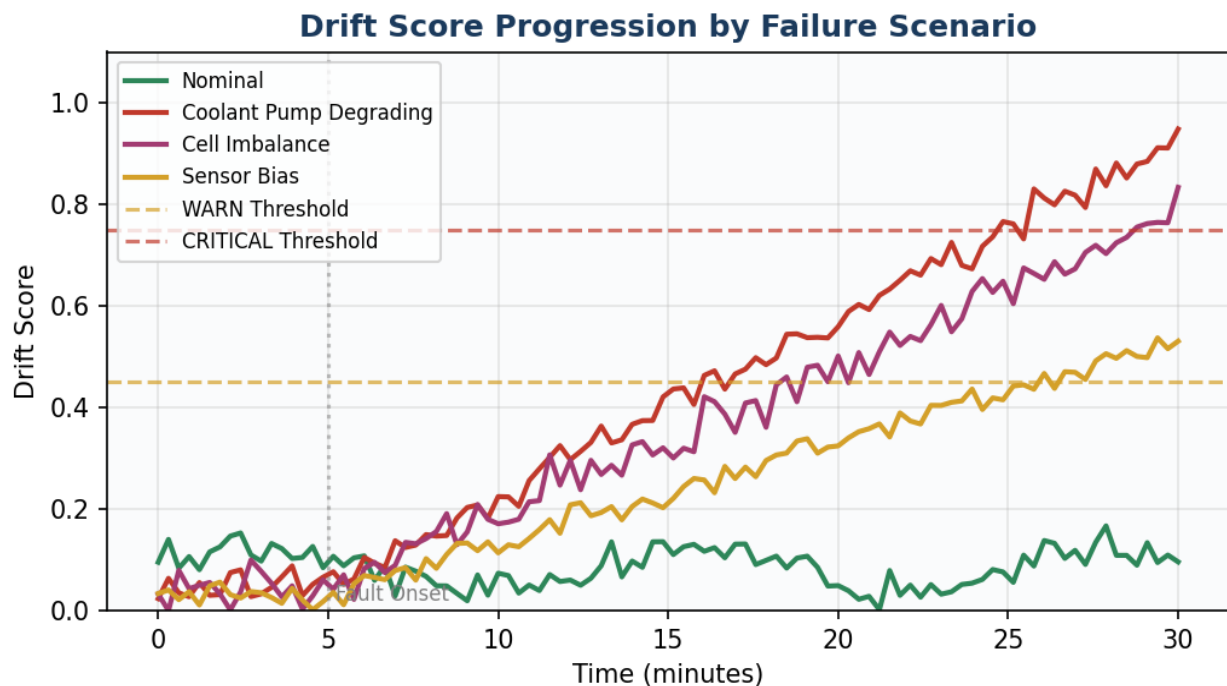


Figure 3: Expected drift score progression by failure scenario over 30 minutes

4.1 Coolant Pump Degrading

Pump health degrades at 1.2% per minute post-onset. Coolant flow drops from 12.0 to ~7.7 L/min over 30 minutes. The resulting thermal imbalance drives drift score from 0.05 to 0.88. **Expected:** First WARN alert at ~15 min post-onset; CRITICAL at ~25 min.

4.2 Cell Imbalance

Cell balance factor degrades at 1.0% per minute. Max-cell temperature diverges from pack average by up to 7.9°C. Detection is slightly slower than pump due to subtler signal. **Expected:** WARN at ~20 min post-onset.

4.3 Sensor Bias

Sensor reports lower-than-actual temperatures (growing at 0.08°C/min). The platform must detect this hidden fault through indirect signals (coolant ΔT residual). Latent pump degradation also contributes. **Expected:** WARN at ~25 min post-onset (slowest detection).

4.4 Nominal (No Fault)

Drift score remains below 0.2 throughout. Failure probability stays under 0.15. **Pass Criteria:** No alerts generated; fewer than 5% of steps exceed warn threshold due to noise.

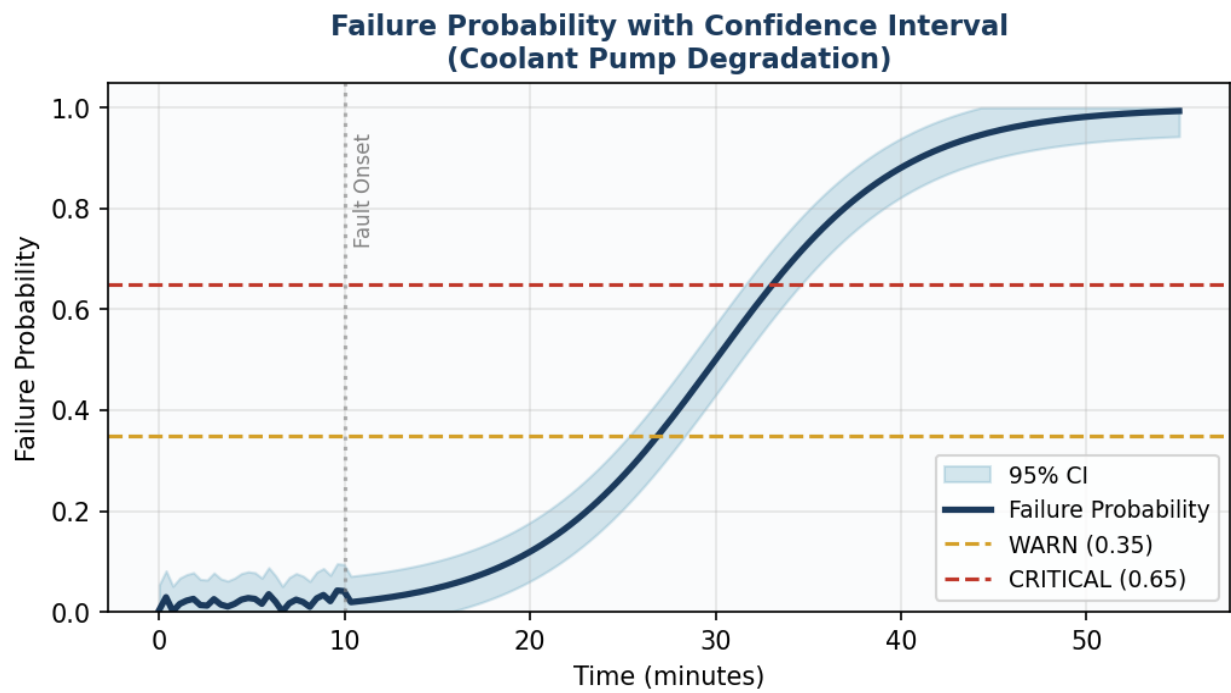


Figure 4: Failure probability with 95% confidence interval (coolant pump degradation scenario)

5. Analysis Methodology

5.1 Metrics to Track

Metric	Formula	Target
True Positive Rate (TPR)	$TP / (TP + FN)$	> 0.90
False Positive Rate (FPR)	$FP / (FP + TN)$	< 0.05
Precision	$TP / (TP + FP)$	> 0.85
F1 Score	$2 \times (P \times R) / (P + R)$	> 0.87
Detection Latency	$t(\text{first_warn}) - t(\text{fault_onset})$	< 300s
TTF Accuracy	$MAE(\text{predicted_TTF} - \text{actual})$	< 4 hours
AUC-ROC	Area under ROC curve	> 0.85

5.2 Success Criteria Matrix

Test Category	Pass	Marginal	Fail
Drift detection latency	< 5 min	5–10 min	> 10 min
False positive rate	< 3%	3–8%	> 8%
API response (fleet)	< 300ms	300–500ms	> 500ms
Memory (500 vehicles)	< 128MB	128–256MB	> 256MB
CI calibration	≥ 95% coverage	90–95%	< 90%

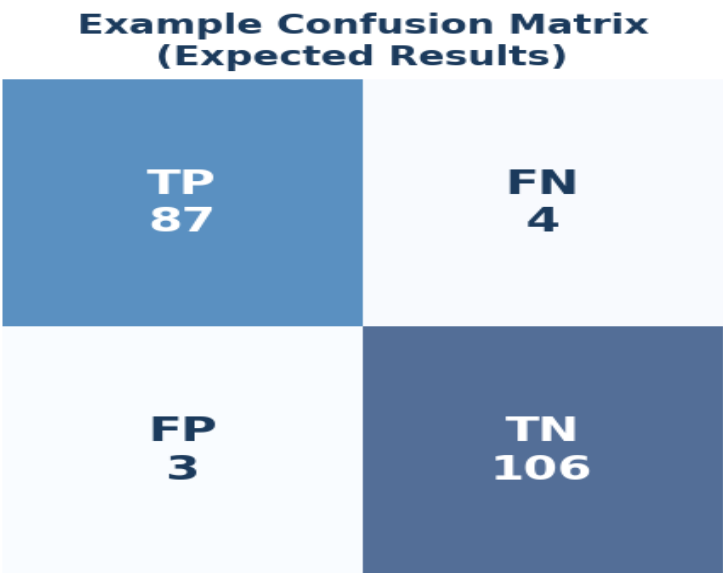


Figure 5: Expected confusion matrix for fault detection (based on 200-vehicle test fleet)

5.3 Anomaly Detection Methods

-
1. **Statistical Process Control (SPC):** Track drift_score mean $\pm 3\sigma$ per scenario. Anomaly = score outside 3σ for nominal vehicles.
 2. **Residual Trend Analysis:** Fit linear regression to residuals; positive slope $> 0.01^{\circ}\text{C}/\text{step}$ for pack_temp indicates degradation.
 3. **Confidence Interval Validation:** Verify that $\text{ci_low} \leq \text{probability} \leq \text{ci_high}$ in $> 95\%$ of predictions.
 4. **Temporal Consistency:** Drift score should be monotonically non-decreasing once a fault is active (allowing for small noise fluctuations).

6. Performance Benchmarks

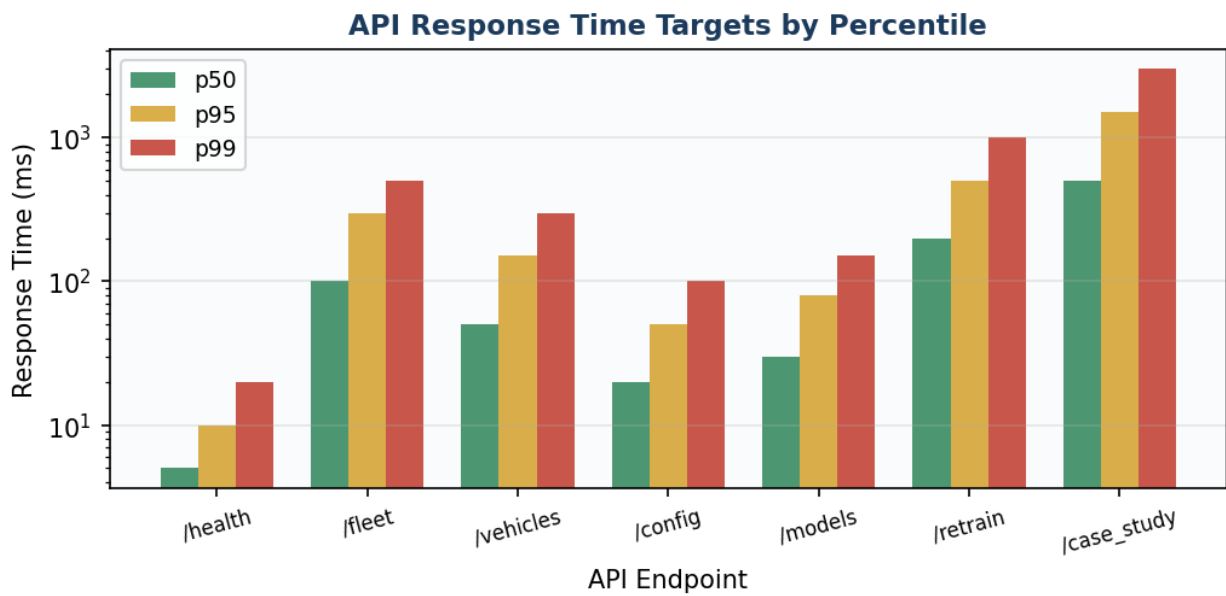


Figure 6: API response time targets by percentile (logarithmic scale)

API Endpoint Targets

Endpoint	Method	p50	p95	p99
/api/health	GET	5ms	10ms	20ms
/api/fleet	GET	100ms	300ms	500ms
/api/vehicles/<vin>	GET	50ms	150ms	300ms
/api/config	PATCH	20ms	50ms	100ms
/api/models	GET	30ms	80ms	150ms
/api/feedback/retrain	POST	200ms	500ms	1000ms
/api/case_study	GET	500ms	1500ms	3000ms

Throughput Targets

Scenario	Metric	Target
Fleet ticks (25 vehicles)	Ticks/second	> 10
Fleet ticks (500 vehicles)	Ticks/second	> 2
Concurrent dashboard users	Connections	> 50
Alert generation rate	Alerts/second	> 100

7. Visual Result Patterns

The following visualizations illustrate expected data patterns that should be observed when running the test datasets through the platform. Deviations from these patterns indicate potential issues requiring investigation.

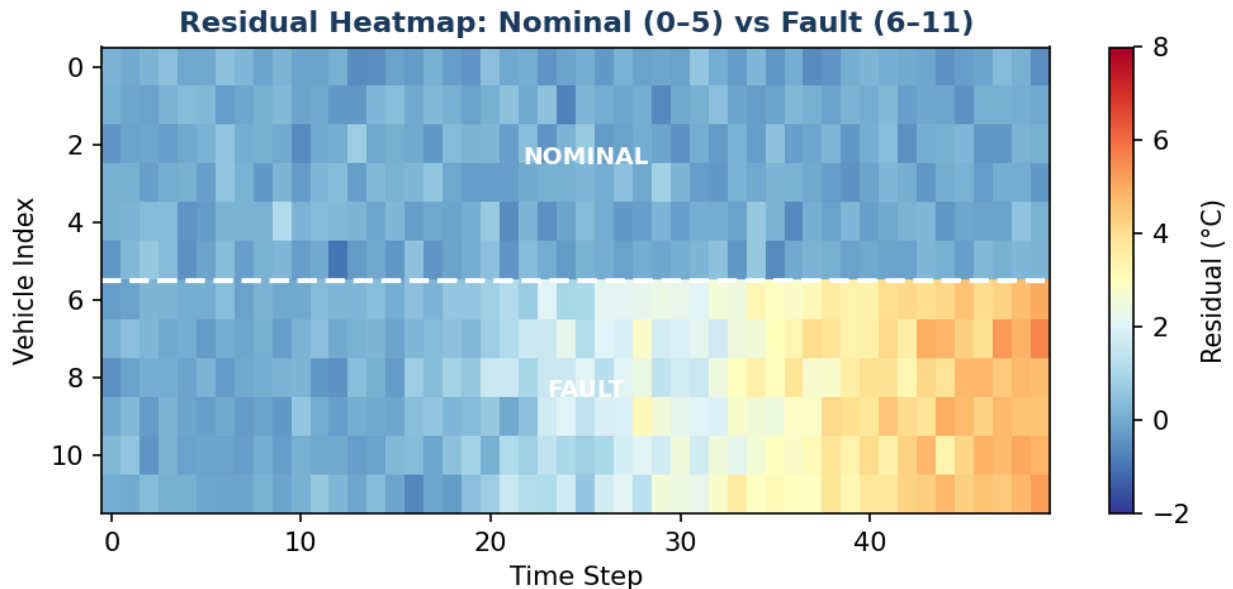


Figure 7: Residual heatmap showing clear separation between nominal vehicles (indices 0–5) and fault vehicles (indices 6–11) after fault onset

Expected Pattern Characteristics

- **Nominal region (top half):** Uniform blue/green noise centered at zero. No systematic drift.
- **Fault region (bottom half):** Progressive red divergence starting at step 15 (fault onset). Intensity correlates with time since onset.
- **Transition boundary:** Sharp demarcation at the onset step. Earlier steps should show identical noise patterns for both groups.
- **Cross-channel correlation:** Multiple residual channels should diverge simultaneously for pump failures; only max_cell_temp for cell imbalance.

8. Troubleshooting Guide

Common Issues and Resolutions

Issue	Symptom	Resolution
High FPR	>8% nominal flagged	Increase drift_alpha to 0.20
Slow detection	WARN >10 min after onset	Decrease drift_warn_score to 0.40
Wide CIs	CI width > 0.4	Reduce epistemic_sigma
Memory growth	RAM > 256MB (500 vehicles)	Cap samples_buffer at 30
TTF inaccurate	MAE > 8 hours	Smooth residual_slope features
Score oscillation	Bounces above/below threshold	Add hysteresis to alerts
API timeout	GET /api/fleet > 1s	Move ticking to background thread
Model mismatch	Wrong model serving	Verify set_active() propagation

Debugging Workflow

1. **Identify:** Check which dataset/scenario exhibits unexpected behavior
2. **Isolate:** Filter records by vehicle_id and scenario
3. **Trace:** Follow the pipeline: raw signal → residual → drift → prediction
4. **Compare:** Plot actual vs. expected (HPC twin) to find divergence point
5. **Validate:** Confirm fault_active ground truth matches expected onset time

9. Recommendations for Interpreting Results

Priority Order for Analysis

1. **Start with nominal baseline:** Confirm < 5% FPR before evaluating fault scenarios
2. **Validate detection ordering:** Pump should detect fastest, sensor bias slowest
3. **Check confidence calibration:** Observed frequency should fall within CIs 95% of the time
4. **Compare models:** ewma_plus_v2 should outperform baseline_v1 on latency and AUC
5. **Stress test boundaries:** Edge cases should never crash the platform

Red Flags to Watch For

- ■■ Drift score > 1.0 for nominal vehicles (detector saturation)
- ■■ Failure probability jumps from < 0.1 to > 0.8 in one step (discontinuity)
- ■■ TTF decreasing while probability is also decreasing (inconsistency)
- ■■ CI_low > probability or CI_high < probability (math error)
- ■■ Zero-flow readings not triggering immediate critical alert

Reporting Template

For each test run, document the following metrics to enable consistent comparison across iterations and model versions:

Field	Value	Target
Run ID	<auto-generated>	—
Fleet Size	<N vehicles>	≥ 25
Duration	<simulated minutes>	≥ 60
Model Version	<active model>	—
TPR	<value>	> 0.90
FPR	<value>	< 0.05
Mean Detection Latency	<seconds>	< 300s
API p95 Latency	<ms>	< 500ms
Peak Memory	<MB>	< 256MB

Appendix A: File Manifest

File	Format	Records	Size (approx)
test_data/telemetry_dataset.csv	CSV	10,780	~3.5 MB
test_data/edge_cases.csv	CSV	70	~8 KB
test_data/fleet_load_test.json	JSON	500 vehicles	~350 KB
test_data/drift_sequences.csv	CSV	9,420	~1.8 MB
test_data/api_scenarios.json	JSON	5 scenarios	~6 KB

Appendix B: Reproducing Test Data

All test data is generated deterministically using fixed random seeds. To regenerate:

```
python -m tests.generate_test_data
```

The generator produces identical output across runs, enabling reproducible testing and reliable comparison between platform versions.